

UNIVERSIDAD NACIONAL PEDRO RUIZ GALLO

Facultad de Ingeniería Civil, de Sistemas y de Arquitectura

Escuela Profesional de Ingeniería de Sistemas

SISTEMA WEB INTERACTIVO DE TEORÍA DE CONJUNTOS Y DIAGRAMAS DE VENN

Informe Técnico Final del Proyecto

INTEGRANTES (EQUIPO LINUS - GRUPO 4):

Jordy Jose (Sublíder)

Junior

Mike

Sergio

Fernando

Alejandro

DOCENTE:

Dr. Mardo Victor Gonzales Herrera

ÍNDICE GENERAL

1. Introducción y Propuesta del Proyecto (Sprint 1).....	Pág. 4
2. Arquitectura del Software y Flujo de Trabajo (Git Flow).....	Pág. 4
3. Capa Lógica: El Motor Matemático (Sprint 2).....	Pág. 5
4. Capa Visual: Interfaz con Vibe Duolingo (Sprint 3).....	Pág. 5
5. Pruebas de Calidad del Código (Sprint 4).....	Pág. 6
6. En qué nos ayudó este proyecto.....	Pág. 6
7. Participación y Aportes del Equipo.....	Pág. 7
8. Conclusiones.....	Pág. 7

1. INTRODUCCIÓN Y PROPUESTA DEL PROYECTO (SPRINT 1)

El aprendizaje de la Teoría de Conjuntos representa una base fundamental en la lógica matemática y la ciencia de la computación. No obstante, la visualización espacial de las operaciones (como uniones, intersecciones complejas y complementos) y el cálculo manual del Conjunto Potencia 2^N suelen ser fuentes de confusión académica.

Para mitigar este problema, el Equipo Linus desarrolló una herramienta de software interactiva y didáctica basada en la web. La funcionalidad se diseñó a partir de bocetos y maquetas preliminares de la interfaz que sirvieron de base para estructurar el software final.

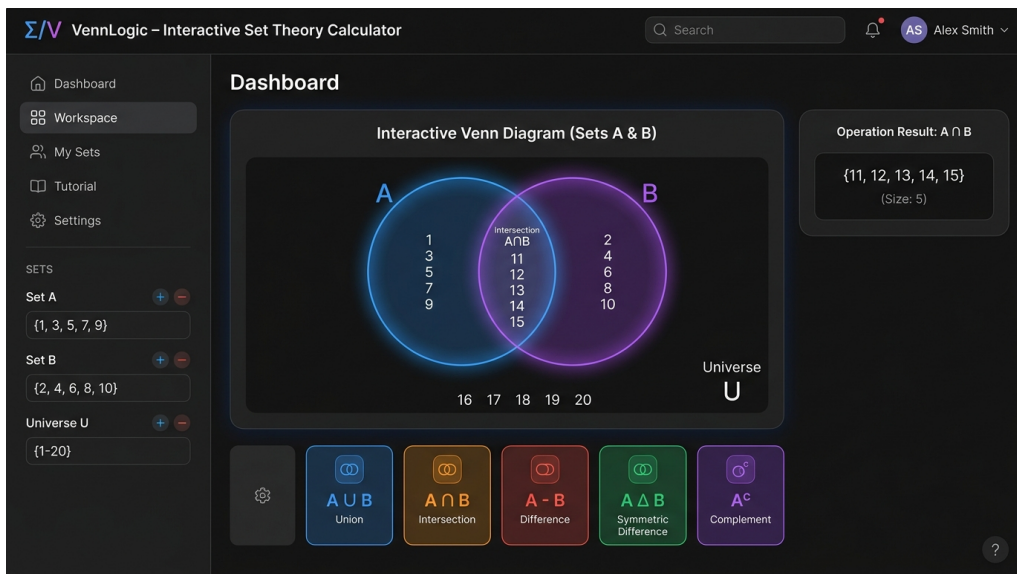


Figura 1: Diseño en maqueta inicial (Wireframe) del proyecto.

2. ARQUITECTURA DEL SOFTWARE Y FLUJO DE TRABAJO (GIT FLOW)

El proyecto se diseñó bajo una arquitectura desacoplada para asegurar escalabilidad y modularidad, separando la lógica pura (el motor) de la interfaz de usuario.

Estructura de Directorios:

- Capa Lógica: `src/lib/sets/operations.ts`
- Pruebas Unitarias: `src/lib/sets/operations.test.ts`
- Capa de Interfaz: `src/pages/conjuntos/index.vue`
- Documentación de Avance: `TAREAS/02_Motor/linus/avance.md`, `TAREAS/03_UI/linus/avance.md`...

Flujo de Git por Ramas:

Para mantener el repositorio limpio, cada fase de desarrollo se desarrolla en una rama de Git independiente antes de enviarse al repositorio principal mediante Pull Requests (PR):

- equipo-4-linus-propuesta (Sprint 1)
- equipo-4-linus-motor (Sprint 2)
- equipo-4-linus-ui (Sprint 3)
- equipo-4-linus-deploy (Sprint 4)

Nombre	Mensaje del último commit	Fecha del último...
...		
01_Propuesta	docs: agregar la propuesta del Sprint 1 y los registros de tareas motoras del Sprint 2	hace 3 días
02_Motor	tarea: fusionar upstream/main en sprint-3-hijos-de-linus	ayer
03_UI	tarea: fusionar upstream/main en sprint-3-hijos-de-linus	ayer
04_Despliegue	tarea: fusionar upstream/main en sprint-3-hijos-de-linus	ayer
ESTADO_EQUIPOS.md	docs: agrega PRs #9, #10, #11 al registro de cambios (Equipo Linus)	anteayer

Figura 2: Historial de desarrollo secuencial por ramas del Equipo Linus.

3. CAPA LÓGICA: EL MOTOR MATEMÁTICO (SPRINT 2)

El motor logico realiza todos los calculos matematicos de conjuntos. Se implemento en TypeScript de forma estricta (tipado estricto) utilizando la estructura de datos nativa Set de ES6.

Ventaja del Tipo Set:

1. Filtran automaticamente elementos repetidos de forma nativa.
2. La busqueda de elementos con `.has()` tiene una complejidad temporal de $O(1)$ (tiempo constante).

Codigo Fuente de `src/lib/sets/operations.ts`:

```
export function union<T>(a: Set<T>, b: Set<T>): Set<T> {
  return new Set([...a, ...b]);
}

export function interseccion<T>(a: Set<T>, b: Set<T>): Set<T> {
  return new Set([...a].filter(x => b.has(x)));
}

export function diferencia<T>(a: Set<T>, b: Set<T>): Set<T> {
  return new Set([...a].filter(x => !b.has(x)));
}

export function complemento<T>(universo: Set<T>, a: Set<T>): Set<T> {
  return diferencia(universo, a);
}

export function potencia<T>(a: Set<T>): Set<Set<T>> {
  const elementos = Array.from(a);
  const subconjuntos = new Set<Set<T>>();
  const numSubconjuntos = Math.pow(2, elementos.length);
  for (let i = 0; i < numSubconjuntos; i++) {
    const subconjunto = new Set<T>();
    for (let j = 0; j < elementos.length; j++) {
      if ((i & (1 << j)) !== 0) {
        subconjunto.add(elementos[j]);
      }
    }
    subconjuntos.add(subconjunto);
  }
  return subconjuntos;
}
```

4. CAPA VISUAL: INTERFAZ CON VIBE DUOLINGO (SPRINT 3)

La pantalla interactiva en index.vue conecta el motor logico con el usuario final de manera atractiva y sencilla, adoptando el diseño de estilo "Duolingo" (color azul de acento, bordes redondeados amplios y tarjetas limpias).

Reactividad en Vue 3:

- ref: Usado para registrar el texto ingresado por el usuario (Inputs del Universo, A y B).
- computed: Propiedades calculadas en memoria con cache de procesamiento. Permiten actualizar el resultado matematico y el diagrama SVG automaticamente en tiempo real solo cuando cambian los inputs de origen.

Iluminacion Dinamica del Diagrama de Venn (SVG):

En lugar de importar una libreria pesada de graficos, se programo un diagrama vectorial SVG nativo. El color de las regiones cambia en tiempo real mediante la clase reactiva vennColores, asignando opacidades al circulo de la izquierda, derecha o a la interseccion segun la operacion seleccionada.

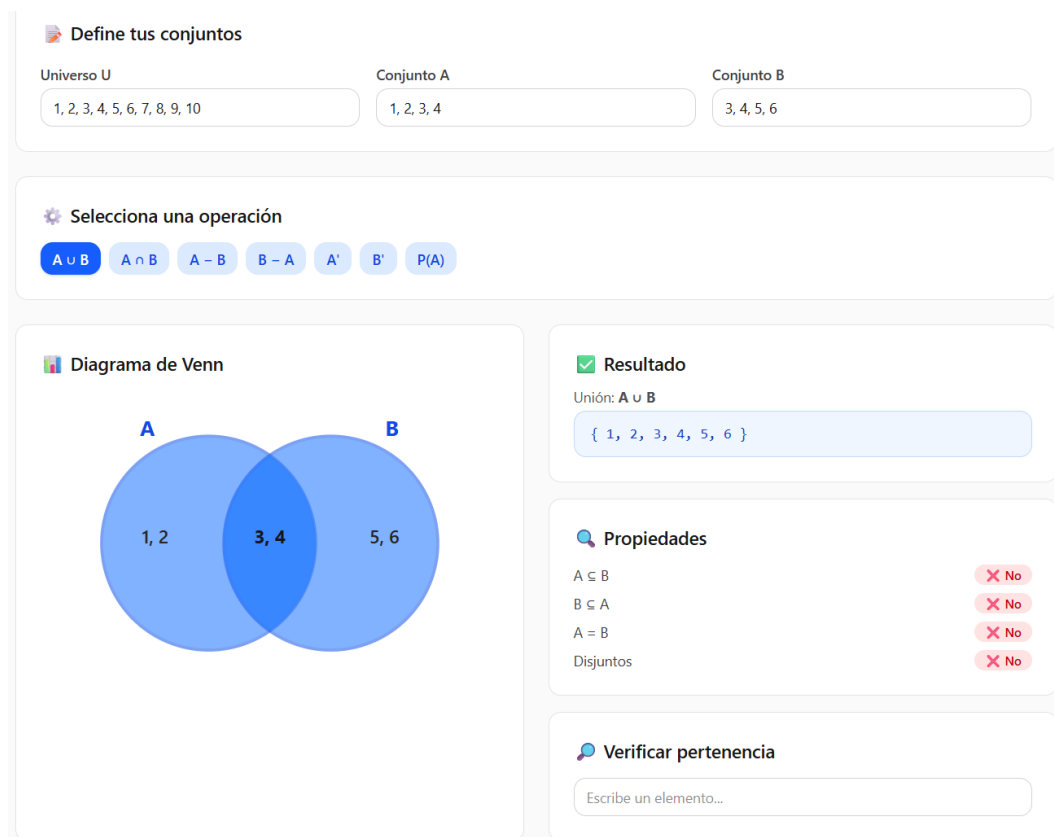


Figura 3: Captura de pantalla de la interfaz interactiva y el Diagrama de Venn en accion.

5. PRUEBAS DE CALIDAD DEL CÓDIGO (SPRINT 4)

Para asegurar que las matematicas de la aplicacion sean infalibles y no presenten errores logicos, se disenó una bateria de 14 pruebas unitarias con la suite de testing Vitest.

Estas pruebas comprueban leyes matematicas fundamentales, tales como:

- Ley conmutativa: $A \cup B = B \cup A$
- Ley distributiva: $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$

6. ¿EN QUÉ NOS AYUDÓ ESTE PROYECTO?

El desarrollo colaborativo de la Calculadora de Conjuntos ha proporcionado al equipo beneficios significativos tanto en el ambito tecnico como formativo:

- Consolidacion Matematica: Traducir axiomas abstractos de la teoria de conjuntos a codigo ejecutable nos brindo un entendimiento profundo del comportamiento logico.
- Metodologia Git Profesional: Aprendimos a trabajar de manera sincronizada en ramas independientes mediante bifurcaciones (Forks) y Pull Requests, emulando la dinamica de equipos de desarrollo reales en la industria.
- Trabajo y Cohesion en Equipo: Fomento la delegacion de roles (desarrollo backend, frontend, pruebas de calidad y documentacion), mejorando nuestras habilidades de comunicacion y resolucion conjunta de conflictos.

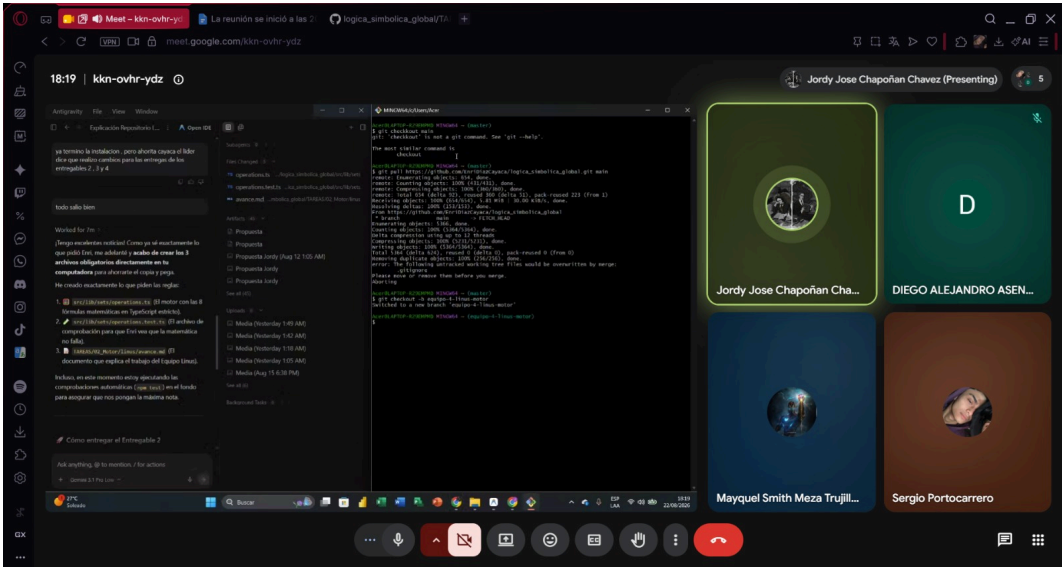


Figura 4: Reunion de coordinacion y trabajo en equipo para la integracion del proyecto.

7. PARTICIPACIÓN Y APORTES DEL EQUIPO

Integrante	Rol Principal	Contribucion Clave
Jordy Jose	Sublider / Git Manager	Gestion de ramas, fusiones y desarrollo de UI.
Junior	Analista de Logica	Verificacion del modulo de propiedades y pertenencia.
Mike	Ingeniero de Pruebas	Implementacion de casos limite en Vitest.
Sergio	Desarrollador Frontend	Implementacion de la visualizacion SVG del diagrama.
Fernando	Documentador Tecnico	Elaboracion del manual de usuario y documentacion.
Alejandro	Disenador de Software	Modelado inicial del comportamiento del sistema.

8. CONCLUSIONES

1. Se logro construir una aplicacion web interactiva que traduce conceptos matematicos abstractos de la teoria de conjuntos en graficos dinamicos comprensibles.
2. La separacion limpia entre la logica matematica (operations.ts) y la presentacion visual (index.vue) demostro ser

clave para el desarrollo colaborativo y la ejecución de pruebas robustas sin acoplamiento gráfico.